Contenido

Introducción 2
Objetivo general 3
Objetivos específicos 3
Marco conceptual 4
Metodología 7
Desarrollo 9
Dataset y estructura de carpetas 9
Verificación de las imágenes 10
Preprocesamiento 10
Reescalado de imágenes 11
Conversión a escala de grises 12
Binarización de imágenes 13
Código del preprocesado 14
Detección de contornos 16
Método de Canny 16
Etiquetado de objetos 18
Construcción del algoritmo RNA 20
Resultados 21
Perros vs gatos 21
Clasificación entre razas 25
Conclusiones 27
Bibliografía 28

Introducción

Recientemente la Inteligencia Artificial ha tenido un crecimiento significativo en múltiples áreas, especialmente en el reconocimiento de patrones y visión por computadora. Dentro de sus aplicaciones más comunes se encuentran la clasificación automática de imágenes, permitiendo identificar objetos, animales, personas o escenarios mediante algoritmos usando una gran cantidad de datos. Dentro de este campo, el reconocimiento de animales como perros y gatos representa un problema interesante debido a la diversidad de formas, tamaños, colores y características físicas existentes entre especies, razas e incluso entre los propios individuos de estas.

El presente proyecto consiste en el desarrollo de un sistema capaz de analizar imágenes digitales para determinar si el animal mostrado corresponde a un perro o a un gato, así como identificar su posible raza. Para lograrlo, se aplican técnicas de procesamiento digital de imágenes, tales como conversión a escala de grises, binarización, detección de contornos y etiquetado de objetos, con el objetivo de extraer información relevante de cada imagen.

Posteriormente, se implementa una Red Neuronal Artificial (RNA), la cual aprende patrones visuales a partir de un conjunto de imágenes previamente clasificadas. Gracias al entrenamiento del modelo, el sistema es capaz de realizar predicciones sobre nuevas imágenes no vistas anteriormente. Este tipo de soluciones demuestra el potencial de la Inteligencia Artificial para automatizar tareas de clasificación visual de forma rápida y eficiente.

Objetivo general

Desarrollar un sistema basado en Inteligencia Artificial capaz de reconocer imágenes de perros y gatos, clasificando la especie del animal e identificando su raza mediante técnicas de procesamiento digital de imágenes y Redes Neuronales Artificiales.

Objetivos específicos

- Recolectar y organizar un conjunto de al menos 250 imágenes de perros y gatos clasificadas por raza.
- Aplicar técnicas de preprocesamiento de imágenes para mejorar la calidad visual de los datos.
- Convertir las imágenes a formato binario para facilitar el análisis estructural.
- Implementar métodos de detección de contornos para identificar la forma del animal dentro de la imagen.
- Realizar el etiquetado de objetos presentes en cada imagen.
- Diseñar y entrenar una Red Neuronal Artificial para clasificar imágenes entre perros y gatos.
- Extender la clasificación para reconocer diferentes razas de cada especie.
- Evaluar el rendimiento del sistema mediante métricas de precisión y exactitud.

Marco conceptual

Inteligencia Artificial

La Inteligencia Artificial es una rama de la informática enfocada en desarrollar sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como aprender, reconocer patrones, tomar decisiones y resolver problemas.

Visión por Computadora

La visión por computadora es un área de la Inteligencia Artificial que permite a las máquinas interpretar imágenes y videos digitales. Su objetivo es extraer información útil del entorno visual.

Procesamiento Digital de Imágenes

Consiste en aplicar operaciones matemáticas y algoritmos sobre imágenes digitales para mejorar su calidad o extraer características relevantes.

Escala de Grises

Es una representación de la imagen donde cada píxel contiene únicamente intensidad luminosa, eliminando la información de color.

Binarización

Proceso mediante el cual una imagen se transforma en solo dos colores: blanco y negro. Se utiliza un umbral para separar objeto y fondo.

Etiquetado de Objetos

Es una técnica que permite identificar regiones conectadas dentro de una imagen binaria, asignando una etiqueta distinta a cada objeto detectado.

Detección de Contornos

Permite localizar los bordes de los objetos dentro de una imagen. Algunos métodos utilizados son:

- Método de Roberts. Utiliza operadores diagonales para detectar cambios bruscos de intensidad.
- Método de Prewitt. Detecta bordes mediante derivadas horizontales y verticales.
- Método de Sobel. Similar a Prewitt, pero da mayor peso a píxeles centrales.
- Método de Canny. Uno de los más precisos, ya que reduce ruido y detecta bordes definidos mediante múltiples etapas.

Red Neuronal Artificial (RNA)

Es un modelo computacional inspirado en el cerebro humano, compuesto por neuronas artificiales conectadas entre sí. Aprende patrones mediante entrenamiento con datos etiquetados.

Dataset

Conjunto de datos utilizado para entrenar y evaluar el modelo. En este proyecto estará conformado por imágenes de perros y gatos clasificadas por raza.

Métricas de Evaluación

Se utilizan para medir el rendimiento del modelo, entre estas se encuentran:

- Precisión: porcentaje de predicciones correctas.
- Exactitud (Accuracy): proporción total de aciertos.
- Matriz de confusión: muestra aciertos y errores por clase.

Librerías usadas en el desarrollo del proyecto

TensorFlow y Keras

La librería TensorFlow es una plataforma de código abierto ampliamente utilizada para el desarrollo de modelos de aprendizaje automático. Dentro de este proyecto, se utilizó su API de alto nivel Keras, la cual facilita la construcción y entrenamiento de redes neuronales mediante una interfaz más sencilla.

Redes Neuronales Convolucionales (CNN)

Las redes neuronales convolucionales son un tipo de red neuronal profunda diseñada para el análisis de datos con estructura espacial, como las imágenes. Estas redes están compuestas por varias capas que permiten extraer características relevantes de forma jerárquica.

Las principales capas utilizadas en este proyecto son:

- Capas convolucionales (Conv2D): Aplican filtros sobre la imagen para detectar características como bordes, texturas y patrones.
- Capas de agrupamiento (MaxPooling): Reducen la dimensionalidad de la imagen, conservando la información más importante.
- Capas densas (Dense): Se encargan de la clasificación final basada en las características extraídas.

Las CNN permiten identificar patrones complejos en imágenes, lo que las hace ideales para tareas como la clasificación de animales por especie o raza.

Normalización de datos

La normalización es un proceso mediante el cual se escalan los valores de entrada a un rango específico, generalmente entre 0 y 1. En este caso, se utilizó una capa de reescalado para dividir los valores de los píxeles entre 255.

Matriz de confusión

La matriz de confusión es una herramienta que permite visualizar el desempeño de un modelo de clasificación. En ella se comparan las etiquetas reales con las predicciones realizadas por el modelo.

Metodología

Para el desarrollo del proyecto se siguieron las siguientes etapas:

1. **Recolección de Datos**

   Se obtuvieron un total de 280 imágenes de perros y gatos. Dichas imágenes están dividas en 35 elementos para cada raza, siendo en total 4 razas de cada categoría. Las imágenes fueron organizadas en carpetas según su clase correspondiente.

2. **Preprocesamiento de Imágenes**

   Cada imagen fue redimensionada a un tamaño uniforme para facilitar su procesamiento. Posteriormente se convirtió a escala de grises y luego a formato binario mediante umbralización.

3. **Segmentación y Análisis**

   Se aplicaron algoritmos de detección de contornos para localizar la silueta principal del animal. Después se realizó el etiquetado de regiones conectadas para identificar objetos dentro de la imagen.

4. **Extracción de Características**

   A partir de las imágenes procesadas se obtuvieron características relevantes como forma general, bordes y distribución de píxeles.

5. **Diseño de la Red Neuronal**

   Se implementó una Red Neuronal Artificial con capas de entrada, capas ocultas y capa de salida. El modelo fue entrenado con imágenes previamente clasificadas.

6. **Entrenamiento y Validación**

   El conjunto de datos fue dividido en datos de entrenamiento y prueba. El modelo aprendió patrones visuales durante varias iteraciones.

7. **Evaluación de Resultados**

   Finalmente se calcularon métricas como precisión, exactitud y matriz de confusión para medir el desempeño del sistema.

Desarrollo

Dataset y estructura de carpetas

Los datasets utilizados para llevar a cabo esta actividad vienen de un recopilado hecho en las páginas de Github y Kaggle, subidas anteriormente por usuarios de la plataforma. Estos incluyen en total 356 razas de perro, cada una con 35 imágenes y 67 razas de gato con cada una teniendo entre 100 y 200 imágenes.

Para propósitos del trabajo, se tomaron en cuenta 4 razas de perro, 4 razas de gato con cada una de estas contando con 35 imágenes. Las razas elegidas fueron las siguientes:

- Perro
    - American foxhound dog (Foxhound americano)
    - Italian pointing dog (Bracco Italiano)
    - Pyrenean sheepdog smooth faced (Pastor de los pirineos de cara rasa)
    - Xoloitzcuintle
- Gato
    - Bengal (Bengala)
    - Raggamuffin
    - Sphynx
    - Turkish angora

Dentro del proyecto, la estructura de carpetas se encuentra de la siguiente manera:

- dataset/
    - cat
        - bengal
        - ragamuffin
        - sphynx
        - turkish_angora

- dog
    - american_foxhound_dog
    - italian_pointing_dog
    - pyrenean_sheepdog_smooth_faced
    - xoloitzcuintle_dog

Por lo que para acceder a cada set de imágenes hay que pasar primeramente por su categoría gato / perro.

Verificación de las imágenes

Para poder verificar que todas las imágenes fueran reconocidas por la librería OpenCV se hizo el siguiente script el cual lee todas las imágenes de una ubicación, mostrándolas en pantalla durante 10 milisegundos en una resolución de 128 por 128 pixeles. No hubo ningún problema al realizar este paso con ninguna de las imágenes.

image_verification.py > ...

```python
import cv2
import os

ruta = "dataset/dog/xoloitzcuintle_dog"

for archivo in os.listdir(ruta):
    img = cv2.imread(os.path.join(ruta, archivo))
    img = cv2.resize(img, (128,128))

    cv2.imshow("Imagen", img)
    cv2.waitKey(10)

cv2.destroyAllWindows()
```

Preprocesamiento

Para el preprocesamiento se llevaron a cabo 4 procesos, de los cuales se mostrará una imagen de ejemplo para ir viendo paso a paso las transformaciones realizadas.

Imagen original.

Reescalado de imágenes

Este paso consiste en redimensionar cada imagen de nuestro dataset para que todas tengan la misma cantidad de información por igual. Este proceso es fundamental en sistemas de visión por computadora, ya que con esto estandarizamos todas las imágenes de entrada.

En términos computacionales, una imagen está compuesta a manera de una matriz de pixeles. Al generar la nueva imagen, se crea otra matriz donde cada posición debe calcularse a partir de los valores originales, esto se realiza principalmente por interpolación.

Para este caso se usó la interpolación bilineal, que calcula el valor de un nuevo pixel como una combinación ponderada de los pixeles vecinos mas cercanos a la imagen original. En opencv, esto se representa mediante la función:

```
img = cv2.resize(img, (128, 128))
```

Donde *img* es la matriz original y *(128, 128)* las dimensiones de la nueva imagen.

Imagen reeescalada.

Conversión a escala de grises

La conversión a escala de grises transforma una imagen en color (generalmente en formato RGB o BGR) a una imagen donde cada píxel representa únicamente la intensidad luminosa.

En una imagen a color, cada píxel tiene tres componentes:

- Rojo (R)
- Verde (G)
- Azul (B)

Para convertir a escala de grises, se calcula una combinación ponderada de estos tres canales, ya que el ojo humano percibe el verde con mayor intensidad que el rojo y el azul.

La fórmula estándar es:

$$Y = 0.299R + 0.587G + 0.114B$$

Donde:

- Y es la intensidad en escala de grises

Esto produce una imagen donde:

- 0 = negro
- 255 = blanco
- valores intermedios = tonos de gris

En OpenCV:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Este proceso reduce la cantidad de información a procesar, eliminando el color y conservando únicamente la estructura visual, lo cual es útil para la detección de bordes.

Imagen a escala de grises.

Binarización de imágenes

La binarización es el proceso mediante el cual una imagen en escala de grises se convierte en una imagen binaria, es decir, donde cada píxel solo puede tomar dos valores:

- 0 (negro)
- 255 (blanco)

Para poder realizar este proceso se utilizó una umbralización adaptativa de la siguiente forma:

$$g(x,y) = \begin{cases} 255 & si\, f(x,y) > T \\ 0 & si\, f(x,y) \leq T \end{cases}$$

Donde:

- f(x,y) es el valor original del píxel
- g(x,y) es el valor binarizado

- T es el valor umbral

A parte, en este método el umbral no es constante, sino que se calcula para cada región de la imagen considerando los píxeles vecinos. Esto permite obtener mejores resultados cuando hay sombras o iluminación variable.

```
_, binary = cv2.threshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                          cv2.THRESH_BINARY, 11, 2)
```

De esta forma primero calculamos el umbral local y luego se ajusta restando una constante.

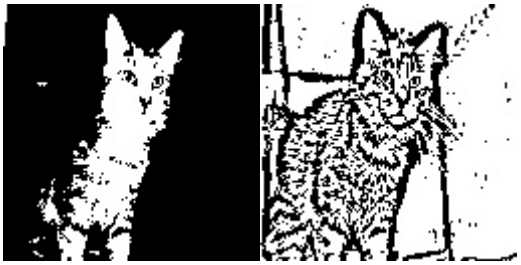


Imagen con umbralización simple (izquierda) y umbralización adaptativa (derecha).

Código del preprocesado

Finalmente, este es el código para esta fase del proyecto en donde escribí el nombre de cada raza separada en un arreglo para gatos y otro para perros, para posteriormente crear una función la cual se encarga de tomar cada imagen de cada carpeta y aplicar las transformaciones vistas anteriormente. Luego de esto las reescribe en la carpeta processed listas para usarse en la RNA.

En resumen, el preprocesamiento de imágenes incluyó el reescalado a un tamaño uniforme, la conversión a escala de grises mediante una combinación ponderada de los canales RGB, y la binarización mediante umbralización, permitiendo separar el objeto del fondo y facilitar el análisis posterior.

preprocess.py > ...

```python
import os
import cv2

cats = ['bengal', 'ragamuffin', 'sphynx', 'turkish_angora']
dogs = ['american_foxhound_dog', 'italian_pointing_dog', 'pyrenean_sheepdog_smooth_faced', 'xoloitzcuintle_dog']

def process_images(input_path, output_path):
    for archive in os.listdir(input_path):
        route = os.path.join(input_path, archive)
        img = cv2.imread(route)

        img = cv2.resize(img, (128, 128))

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        binary = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                       cv2.THRESH_BINARY, 11, 2)

        os.makedirs(output_path, exist_ok = True)

        cv2.imwrite(os.path.join(output_path, archive), binary)


for i in range(4):
    process_images('dataset/cat/' + cats[i], 'processed/cat/' + cats[i])
    process_images('dataset/dog/' + dogs[i], 'processed/dog/' + dogs[i])
```

Detección de contornos

La detección de contornos es una técnica del procesamiento digital de imágenes cuyo objetivo es identificar los bordes de los objetos dentro de una imagen. Un borde corresponde a un cambio brusco en la intensidad de los píxeles, lo cual generalmente indica el límite entre dos regiones, como el objeto y el fondo.

Los contornos son fundamentales porque permiten describir la forma de los objetos, lo que facilita su reconocimiento y clasificación.

De manera general, todos los métodos se basan en detectar cambios de intensidad basados en gradientes, es decir, cuando hay un cambio muy brusco de color o iluminación entonces hay un borde. El gradiente mide que tanto cambia la intensidad de una dirección:

$$G = \sqrt{G_x^2 + G_y^2}$$

Donde:

- Gx: cambio horizontal
- Gy: cambio vertical

Si el gradiente es grande, entonces hay un borde.

Método de Canny

Decidimos usar este algoritmo porque resulta ser el más preciso y completo para detección de bordes. Este método consiste en principalmente 4 etapas:

- Suavizado. Elimina el ruido antes de detectar bordes.
- Cálculo del gradiente. Se calcula la intensidad del borde similar al método de Sobel (Detecta bordes en direcciones horizontal y vertical usando máscaras de 3×3, dando mayor peso a los píxeles centrales).

- Supresión de no máximos. Se eliminan pixeles que no son bordes reales.
- Umbralización con histéresis. Se usa un umbral alto para encontrar los bordes seguros y un umbral bajo para encontrar los bordes más débiles para evitar bordes falsos.

Detección de bordes por el método de Canny luego de aplicar un desenfoque gaussiano.

El único inconveniente al usar este método es que nos vamos a “saltar” el paso de binarización, ya que Canny se beneficia de información en nuestra escala de grises, cosa que perdemos al usar la umbralización simple o adaptativa. Esto no quiere decir que se dejará de usar la binarización, se usará en el etiquetado de objetos, solo que para Canny usaremos directamente la imagen en escala de grises.

Debido a esto, se modificó el método de preprocesado para ahora también contemplar la detección de bordes modificando el método proccess_images:

```
def process_images(input_path, output_path):
    for archive in os.listdir(input_path):
        route = os.path.join(input_path, archive)
        img = cv2.imread(route)

        img = cv2.resize(img, (128, 128))

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        binary = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                       cv2.THRESH_BINARY, 11, 2)

        blur = cv2.GaussianBlur(gray, (5, 5), 0)

        edges = cv2.Canny(blur, 100, 200)

        os.makedirs(output_path, exist_ok = True)

        cv2.imwrite(os.path.join(output_path, archive), edges)
```

Etiquetado de objetos

El etiquetado de objetos se realizó mediante el algoritmo de componentes conectados, el cual permite identificar regiones independientes dentro de una imagen binaria. Cada grupo de píxeles conectados fue asignado a una etiqueta única, facilitando la identificación de objetos presentes en la imagen.

Para mejorar la detección, se utilizó la función que además proporciona estadísticas de cada región, como área, posición y dimensiones. Esto permitió filtrar regiones pequeñas consideradas ruido y resaltar únicamente los objetos relevantes mediante el uso de cajas delimitadoras (bounding boxes).

Este proceso es fundamental para segmentar el objeto de interés y extraer características que posteriormente pueden ser utilizadas en la clasificación mediante redes neuronales.

Código para el ejemplo anterior:

```python

num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(binary_adaptive)

output = reescaled.copy()

for i in range(1, num_labels):
    area = stats[i, cv2.CC_STAT_AREA]
    print(area)

    if area >= 500:
        x = stats[i, cv2.CC_STAT_LEFT]
        y = stats[i, cv2.CC_STAT_TOP]
        w = stats[i, cv2.CC_STAT_WIDTH]
        h = stats[i, cv2.CC_STAT_HEIGHT]

        cv2.rectangle(output, (x, y), (x + w, y + h), (0, 255, 0), 1)

if labels.max() > 0:
    label_img = np.uint8(255 * labels / labels.max())
else:
    label_img = labels

cv2.imwrite('example/labels.jpg', output)
```

Resultado del etiquetado de objetos con área mínima mayor a 0 (izquierda) y área mínima mayor a 500 (derecha). Se observa como con una mayor tolerancia, se detecta ruido como si fueran objetos.

El proceso de etiquetado se hizo para cada imagen del dataset, a la par de la detección de bordes. En el repositorio se pueden observar las demás imágenes del dataset, tanto procesadas con Canny como etiquetadas.

Construcción del algoritmo RNA

El modelo fue entrenado utilizando un conjunto de imágenes dividido en 80% para entrenamiento y 20% para validación. Se empleó una arquitectura de red neuronal convolucional (CNN) con tres capas convolucionales y una capa densa final para clasificación multiclase. Todo esto se logró usando la librería tensorflow.

train.py > ...

```python
import tensorflow as tf

img_size = (128, 128)
batch_size = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    'dataset',
    validation_split = 0.2,
    subset = 'training',
    seed = 123,
    image_size = img_size,
    batch_size = batch_size
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    'dataset',
    validation_split = 0.2,
    subset = 'validation',
    seed = 123,
    image_size = img_size,
    batch_size = batch_size
)

normalization_layer = tf.keras.layers.Rescaling(1./255)

train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))

model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation = 'relu', input_shape = (128, 128, 3)),
    tf.keras.layers.MaxPooling2D(),

```

```python
    tf.keras.layers.Conv2D(64, (3, 3), activation = 'relu'),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(128, (3, 3), activation = 'relu'),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Flatten(),

    tf.keras.layers.Dense(128, activation = 'relu'),
    tf.keras.layers.Dense(8, activation = 'softmax')
])

model.compile(
    optimizer = 'adam',
    loss = 'sparse_categorical_crossentropy',
    metrics = ['accuracy']
)

history = model.fit(
    train_ds,
    validation_data = val_ds,
    epochs = 10
)

loss, acc = model.evaluate(val_ds)
print('presición: ', acc)
```

Este código nos mostrará en consola que tal le fue al modelo en cada época y como es que esta termina mejorando. Finalmente nos arrojará el valor de la precisión y una grafica contrastando el valor de las clases junto a su entrenamiento y validación.

Resultados

Perros vs gatos

El modelo de red neuronal convolucional fue entrenado utilizando un conjunto de imágenes de perros y gatos distribuidas en diferentes razas. Se utilizó una partición del 80% para entrenamiento y 20% para validación.

Como resultado, el modelo alcanzó una precisión del 78.57% sobre el conjunto de validación. Este valor indica que el modelo logra clasificar correctamente aproximadamente 7 de cada 10 imágenes.

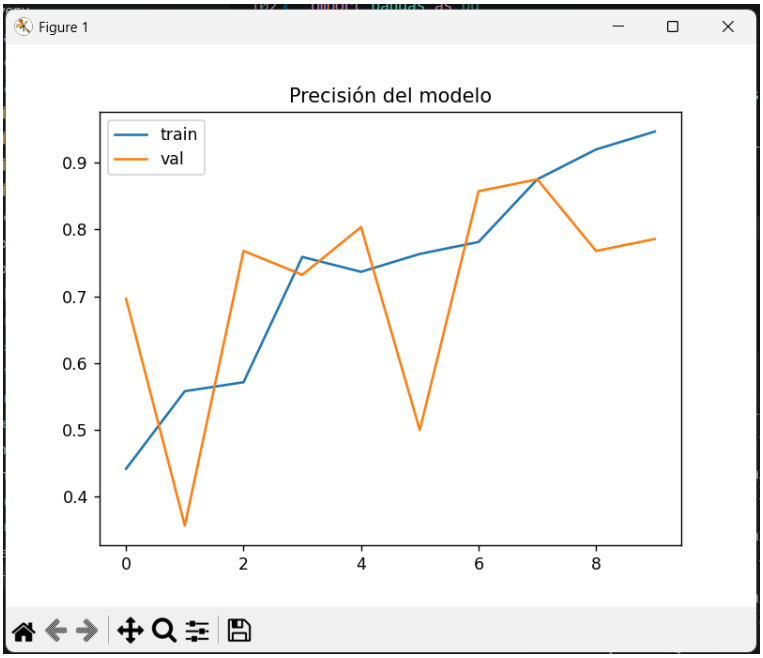
```
(.venv) PS C:\Users\juang\Documents\Escuela\semestre 6\Inteligencia Artificial\tareas\parcial_3> python train.py
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 151ms/step - accuracy: 0.7589 - loss: 0.5386 - val_accuracy: 0.7321 - val_loss: 0.5092
Epoch 5/10
W0000 00:00:1778011792.345950     208 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 147ms/step - accuracy: 0.7366 - loss: 0.5459 - val_accuracy: 0.8036 - val_loss: 0.4751
Epoch 6/10
W0000 00:00:1778011793.490744    6496 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 145ms/step - accuracy: 0.7634 - loss: 0.4965 - val_accuracy: 0.5000 - val_loss: 0.6583
Epoch 7/10
W0000 00:00:1778011794.702728   31140 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 144ms/step - accuracy: 0.7812 - loss: 0.4422 - val_accuracy: 0.8571 - val_loss: 0.4910
Epoch 8/10
W0000 00:00:1778011795.745866    1700 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 146ms/step - accuracy: 0.8750 - loss: 0.3051 - val_accuracy: 0.8750 - val_loss: 0.4257
Epoch 9/10
W0000 00:00:1778011796.963640   29908 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 142ms/step - accuracy: 0.9196 - loss: 0.1988 - val_accuracy: 0.7679 - val_loss: 0.5060
Epoch 10/10
W0000 00:00:1778011798.163555   14020 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 145ms/step - accuracy: 0.9464 - loss: 0.1414 - val_accuracy: 0.7857 - val_loss: 0.5369
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.7857 - loss: 0.5369
precisión:  0.7857142686843872
```

Durante el proceso de entrenamiento, se registró la precisión tanto en el conjunto de entrenamiento como en el de validación a lo largo de las distintas épocas.

La gráfica de precisión muestra que:

- La precisión en entrenamiento incrementa progresivamente, lo que indica que el modelo aprende patrones del conjunto de datos.
- La precisión en validación se mantiene inferior a la de entrenamiento, lo cual sugiere una capacidad limitada de generalización.

Este comportamiento es característico cuando se trabaja con conjuntos de datos pequeños.

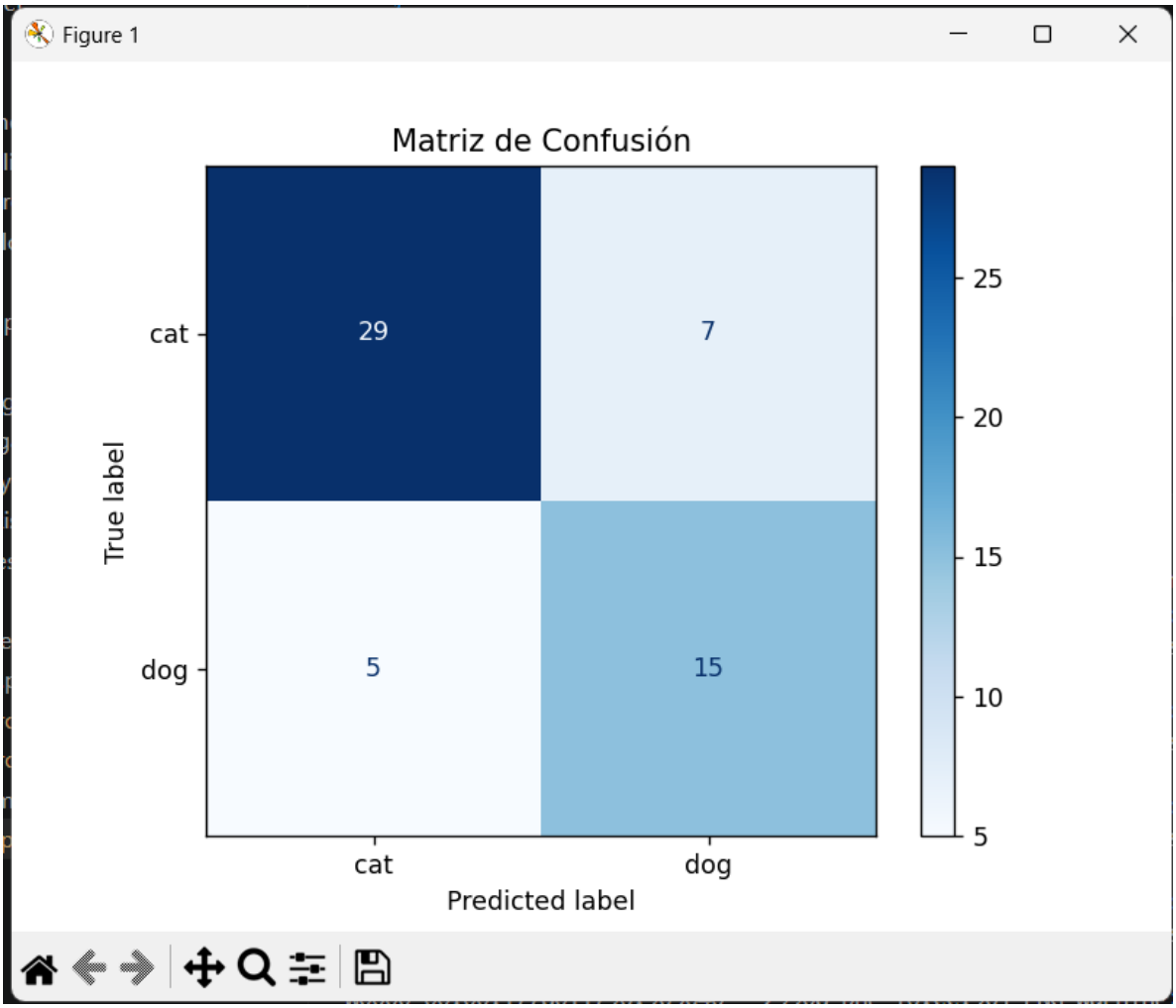


Adicionalmente, se generó una matriz de confusión para analizar el desempeño del modelo por clase. En esta matriz, los valores en la diagonal principal representan las clasificaciones correctas, mientras que los valores fuera de la diagonal indican errores.

El análisis de la matriz permitió identificar que:

- Algunas clases son clasificadas correctamente con mayor frecuencia.
- Existen confusiones entre ciertas razas, especialmente aquellas con características visuales similares.

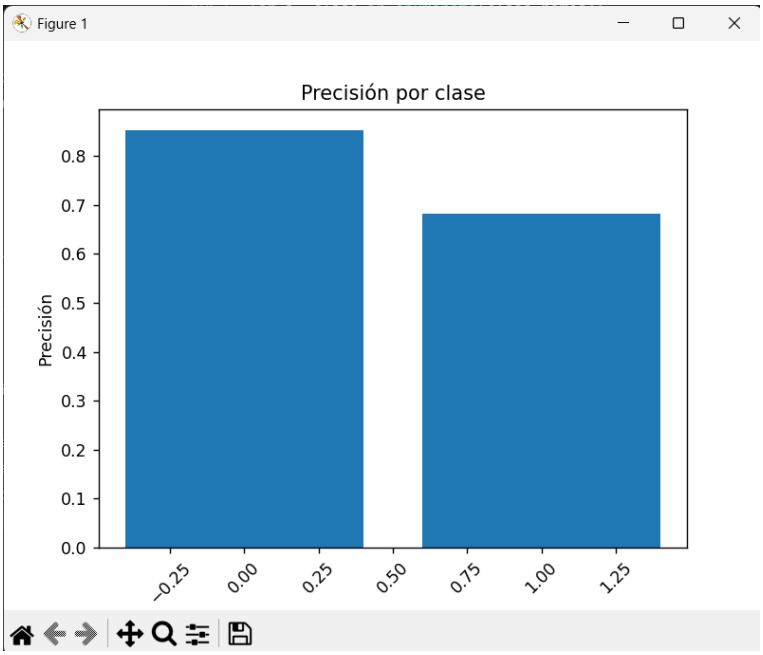
Esto evidencia que el modelo tiene dificultad para distinguir entre clases con rasgos parecidos.



Se evaluó la precisión individual de cada clase, observando variaciones en el desempeño del modelo dependiendo de la raza.

Los resultados muestran que:

- Algunas clases presentan una precisión alta, lo que indica que poseen características distintivas claras.
- Otras clases tienen menor precisión, lo que puede deberse a similitud visual o a menor representatividad en el dataset.



Gráfica de gatos vs perros, se puede observar que la primera clase obtuvo una mejor clasificación para sus elementos, con un 0.88 de precisión.

	precision	recall	f1-score	support
cat	0.88	0.78	0.82	36
dog	0.67	0.80	0.73	20
accuracy			0.79	56
macro avg	0.77	0.79	0.78	56
weighted avg	0.80	0.79	0.79	56

Los resultados obtenidos están influenciados por diversos factores:

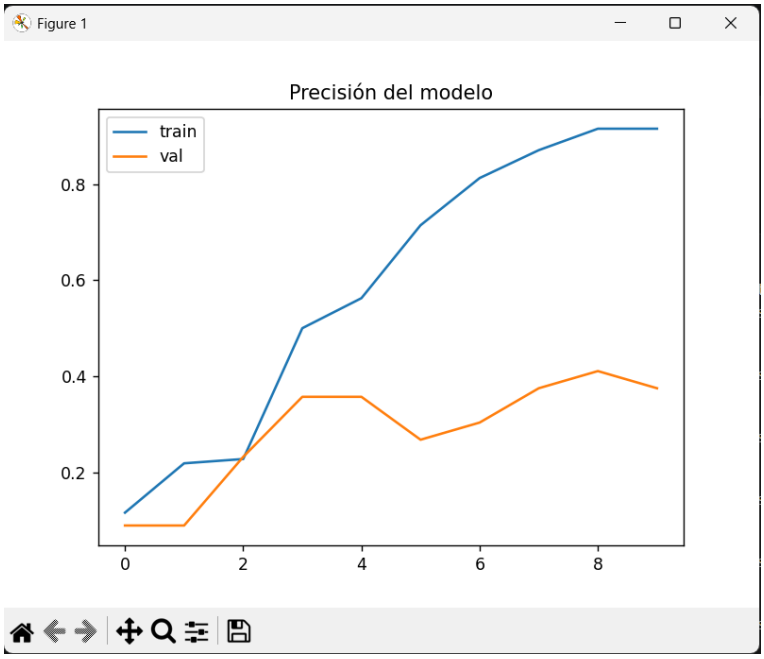
- Cantidad limitada de datos (aproximadamente 35 imágenes por clase)
- Variaciones en iluminación, fondo y calidad de las imágenes
- Similitud entre algunas razas
- Posible sobreajuste debido a la poca cantidad de ejemplos

Clasificación entre razas

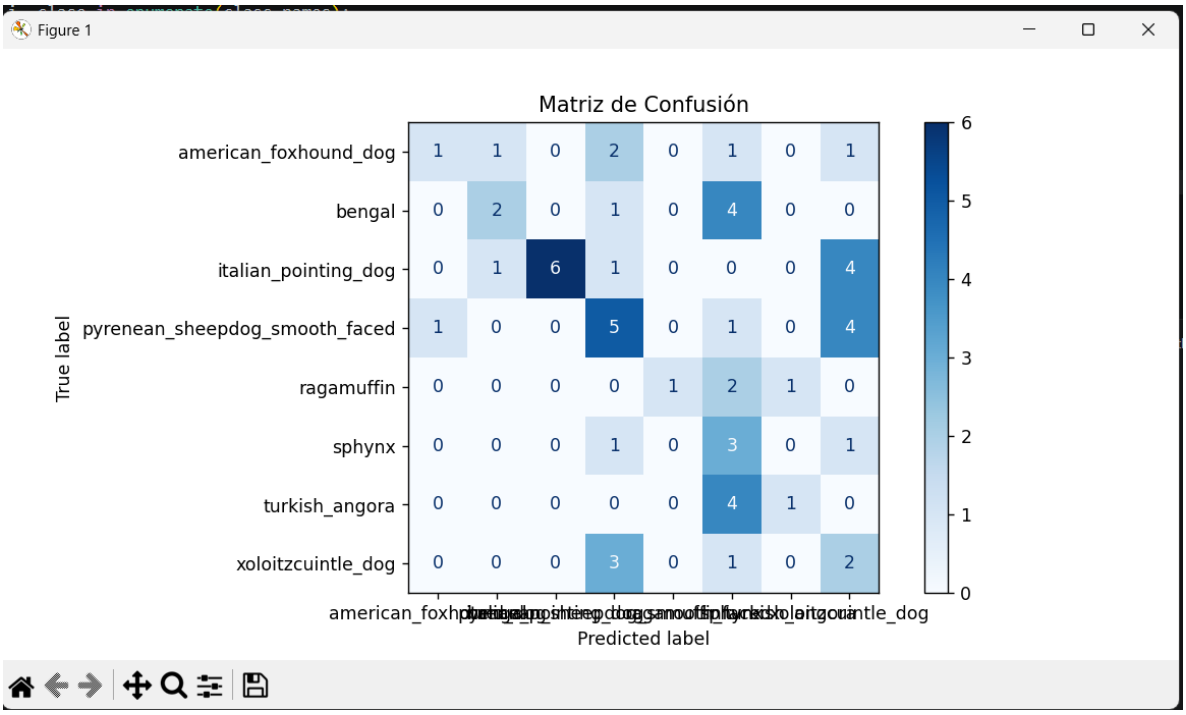
Anteriormente el algoritmo solo clasifico de manera binaria si x imagen pertenecía a la clase perro o la clase gato, ahora cambiaré la estructura del dataset para que evalúe las distintas razas entre sí.

En este caso, al algoritmo le costó más clasificar los elementos ya que obtuvo una puntuación de 0.375, o sea, solo clasificó bien 3 de cada 10 imágenes.

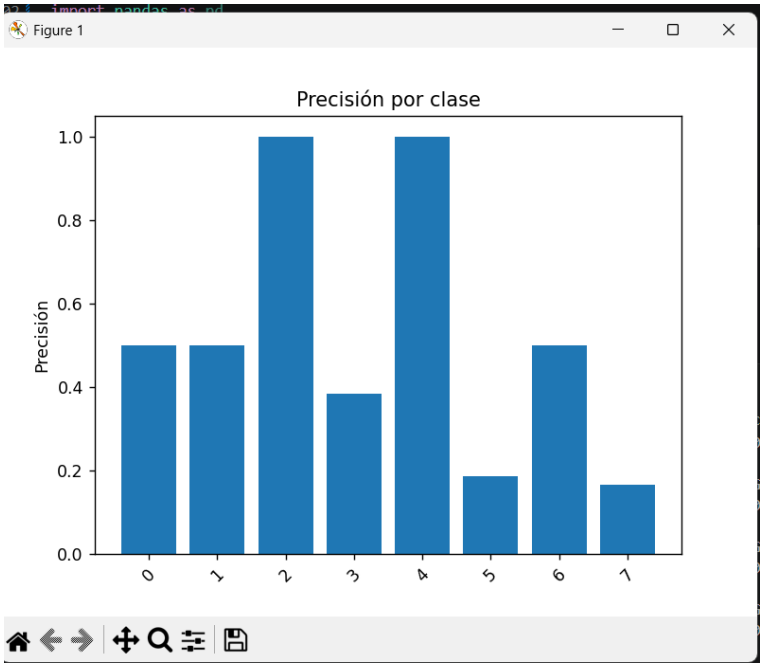
```
(.venv) PS C:\Users\juang\Documents\Escuela\semestre 6\Inteligencia Artificial\tareas\parcial_3> python train.py
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 150ms/step - accuracy: 0.5000 - loss: 1.6564 - val_accuracy: 0.3571 - val_loss: 1.7813
Epoch 5/10
W0000 00:00:1778012835.846853   24328 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 146ms/step - accuracy: 0.5625 - loss: 1.3541 - val_accuracy: 0.3571 - val_loss: 1.7070
Epoch 6/10
W0000 00:00:1778012836.996330    8080 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 145ms/step - accuracy: 0.7143 - loss: 0.9612 - val_accuracy: 0.2679 - val_loss: 2.1430
Epoch 7/10
W0000 00:00:1778012838.025745   29596 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 146ms/step - accuracy: 0.8125 - loss: 0.5951 - val_accuracy: 0.3036 - val_loss: 2.2018
Epoch 8/10
W0000 00:00:1778012839.125129   18000 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 147ms/step - accuracy: 0.8705 - loss: 0.4303 - val_accuracy: 0.3750 - val_loss: 2.0303
Epoch 9/10
W0000 00:00:1778012840.257673    2512 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 149ms/step - accuracy: 0.9152 - loss: 0.3117 - val_accuracy: 0.4107 - val_loss: 2.5575
Epoch 10/10
W0000 00:00:1778012841.469483   30004 png_io.cc:96] PNG warning: iCCP: known incorrect sRGB profile
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 151ms/step - accuracy: 0.9152 - loss: 0.2662 - val_accuracy: 0.3750 - val_loss: 2.3176
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.3750 - loss: 2.3176
precisión:  0.375
```



De igual forma, a la hora de validar el entrenamiento, se nota que claramente le cuesta al modelo diferenciar entre cada raza de perro y gato, posiblemente por el tamaño chico del dataset y problemas de diferenciación del objeto en las imágenes elegidas.



En la matriz de confusión podemos ver que la única raza en la cual se pudo identificar más animales de su misma especie fue con el italian pointing dog, seguido del pyrenean sheepdog. Todas las demás se clasificaron de una forma más inconsistente. Por parte de los gatos, el ragamuffin fue el más sencillo de identificar para su especie.



Precisamente, el pointing dog (2) y el raggamuffin (4) terminan siendo las clases con mayor clasificación para este algoritmo y dataset.

Conclusiones

En este proyecto se desarrolló un sistema de reconocimiento de imágenes capaz de clasificar animales utilizando técnicas de procesamiento digital y redes neuronales convolucionales. Se abordaron dos enfoques: una clasificación binaria (perro vs gato) y una clasificación multiclase por raza.

Los resultados obtenidos muestran una diferencia significativa en el desempeño de ambos modelos. La clasificación binaria alcanzó una precisión de 78%, lo cual indica que el modelo logra distinguir de manera adecuada entre perros y gatos. Por otro lado, la clasificación por razas obtuvo una precisión de 37.5%, evidenciando una mayor dificultad en esta tarea.

Esta diferencia se debe principalmente a la complejidad del problema. Mientras que la clasificación binaria requiere identificar características generales entre dos categorías claramente diferenciadas, la clasificación por razas implica distinguir entre múltiples clases con características visuales similares. Además, el número limitado de imágenes por clase (aproximadamente 35) afecta directamente la capacidad del modelo para generalizar correctamente.

Los resultados confirman que el desempeño de las redes neuronales depende en gran medida de la cantidad y calidad de los datos disponibles, así como de la complejidad del problema a resolver.

A pesar de las limitaciones, el modelo demostró ser funcional y capaz de aprender patrones visuales básicos, cumpliendo con los objetivos planteados. Asimismo, se identificaron áreas de mejora, como el incremento del dataset, el uso de técnicas de aumento de datos y la implementación de modelos más avanzados.

En conclusión, el proyecto evidencia la efectividad de las redes neuronales en tareas de clasificación de imágenes, así como la importancia de contar con un conjunto de datos adecuado para obtener resultados más precisos.

Bibliografía

- MA7555. (2019). *Cat breeds dataset* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/ma7555/cat-breeds-dataset
- GitHub

  AtharvaTaras. (2023). *Dog-breeds-dataset* [Conjunto de datos]. GitHub. https://github.com/atharvataras/dog-breeds-dataset
- Rivera, J. A. (2026). *rna_perros_gatos* [Repositorio de código]. GitHub. https://github.com/JuanAngelRivera/rna_perros_gatos